

DOCUMENTAÇÃO DE PRODUTO · FLYSOP / SOPADMIN

Central Inteligente de Ocorrências Públicas

Guia completo das implementações realizadas: infraestrutura, módulos de negócio, dashboard operacional, despacho inteligente, GPS em tempo real, relatórios e conformidade LGPD.

Projeto: flySOP — Sistema de Ocorrências Públicas

Base técnica: Laravel 8 · PHP 8.1 · PostgreSQL · Redis · Sanctum · AdminLTE · Vue 2 · Google Maps · Pusher/Socket.io

Plano de referência: virtual-jingling-cookie.md

Status deste documento: descreve o sistema *como entregue* (fases 0–6 concluídas)

Data: Agosto/2026

1. Sumário

1. Visão geral do sistema
2. Arquitetura e padrões
3. Mapa de módulos
4. Fase 0 — Segurança e infraestrutura
5. Fase 1 — Fundação de dados da ocorrência
6. Fase 2 — RBAC, departamentos e equipes
7. Fase 3 — Status, auditoria, evidências e notificações
8. Fase 4 — Dashboard, mapa e busca global
9. Fase 5 — Despacho inteligente e GPS robusto
10. Fase 6 — Relatórios, duplicidade e LGPD
11. Passo a passo do ambiente
12. Passo a passo operacional (uso do sistema)
13. Checklist de verificação

2. Visão geral do sistema

O flySOP evoluiu de um painel de ocorrências multi-tenant para uma **Central Inteligente de Ocorrências Públicas**: múltiplos perfis, protocolo e SLA, timeline auditável, mapa operacional com clustering/heatmap, despacho por proximidade, GPS em tempo real, notificações assíncronas, exportações e rotinas LGPD.

Princípio de entrega: evolução do que já existia (Controller fino → Service → Repository), sem redesenhar do zero e sem expandir multi-tenant.

2.1 Perfis de usuário entregues

Perfil	Papel no sistema	Área principal
Administrador	Configuração, usuários, prioridade, status, relatórios e LGPD	AdminLTE completo
Secretaria / Supervisor	Triagem, despacho, acompanhamento de SLA e mapa	Admin filtrado por permissão
Atendente / Operador	Cadastro e atualização de ocorrências, evidências	Admin filtrado
Agente de Campo / Motorista	Aceite, deslocamento, atendimento, GPS e evidências	Painel do motorista
Cidadão	Abertura e consulta de ocorrência (quando autenticado/público controlado)	Site / API

2.2 Capacidades entregues (resumo)

Protocolo + SLA

prioridade e prazo por tipo

Timeline

histórico append-only

Dashboard

KPIs + Chart.js

Mapa

cluster + heatmap

Despacho

ranking por distância

Realtime

WebSocket + fallback

Export

PDF / Excel via fila

LGPD

retenção e esquecimento

3. Arquitetura e padrões

3.1 Stack

Camada	Tecnologia
Backend	Laravel 8, PHP 8.1, Sanctum, Telescope
Painel admin	Blade + AdminLTE + jQuery + Vue 2 (ilhas)
Painel motorista	Blade + JS de geolocalização
Site público	Blade + Vite
Banco	PostgreSQL 15 (principal) + MySQL auxiliar no compose
Fila / cache	Redis + <code>queue:work</code>
Realtime	Pusher ou Soketi (<code>BROADCAST_DRIVER=pusher</code>)
Mapas	Google Maps + MarkerClusterer + HeatmapLayer
Deploy local	Docker Compose (<code>docker-compose.dev.yml</code>) no WSL

3.2 Fluxo padrão de uma request

```
Browser / App
→ routes/web.php | routes/api.php
→ Middleware (auth / auth:sanctum / ensure.driver / can:*)
→ Controller fino
→ FormRequest
→ Service (regra de negócio)
→ RepositoryInterface → Repository → Model (TenantScope)
→ Blade / Resource JSON / Evento broadcast / Job de fila
```

3.3 Quando usar Service/Repository

- **Camadas completas** (Occurrence, Tenant, DriverPosition, Dispatch, Dashboard, Report): regra de negócio, side effects, integrações.
- **CRUD simples** (Priority, Department, Product etc.): controller + model + FormRequest, sem forçar camadas extras.

4. Mapa de módulos implementados

4.1 Ocorrências

Core Domínio central da central inteligente.

- **Camadas:** `OccurrencesController`, `OccurrenceService`, `OccurrenceRepository`, `OccurrenceResource`
- **Campos-chave:** `protocol`, `priority_id`, `due_at`, endereço estruturado, status expandido, `driver_id`, `team_id`
- **Regras:** geração de protocolo, cálculo de SLA, transições de status validadas, histórico append-only, sincronização com status do agente

4.2 Prioridades e tipos

- **priorities:** `name`, `weight`, `color`, `default_sla_hours` — CRUD admin
- **type_occurrences:** `sla_hours` + `parent_id` (tipo/subtipo)
- Prioridade da ocorrência pode herdar SLA do tipo ou da prioridade selecionada

4.3 Status e timeline

- Fluxo: Recebida → Triagem → Aguardando aceitação → Aceita/Recusada → Em deslocamento → Em atendimento → Concluída → Finalizada/Cancelada/Duplicada
- `occurrence_status_history` registra `from/to`, usuário, nota e timestamp
- Flags `is_terminal` e `order` nos status

4.4 RBAC / Departamento / Equipe

- Roles seedadas: Administrador, Supervisor, Atendente, Agente de Campo (Motorista), Cidadão
- Políticas Laravel nativas sobre Role/Permission existentes (sem Spatie Permission)
- `isAdmin()` baseado em Role Administrador (não mais allowlist de e-mail)
- Tabelas `departments` e `teams`; `team_id` opcional em `drivers`

4.5 Motorista / Agente de Campo + GPS

- Tabela `drivers` unificada (Agente de Campo)
- `DriverPositionService` único ponto de gravação de posição
- Endpoints autenticados com Sanctum + throttle
- Evento `DriverPositionUpdated` via broadcasting
- Retenção de posições (> 48h) via job agendado

4.6 Dashboard e mapa operacional

- KPIs: abertas, em atendimento, finalizadas hoje, SLA em risco/estourado
- Gráficos Chart.js
- Google Maps com MarkerClusterer e HeatmapLayer
- Filtros por status, tipo, prioridade, data, agente/equipe
- Push realtime com polling como fallback

4.7 Despacho inteligente

- `DriverRepository` com ranking haversine (Postgres)
- Filtro por disponibilidade e especialidade (`team_id`)
- Sugestão ranqueada; confirmação humana obrigatória

4.8 Notificações, evidências e auditoria

- Notificações database: atribuída, status mudou, SLA em risco
- Evidências antes/depois em `occurrences_imagens` (phase, geo, captured_at, uploaded_by)
- Auditoria de mudanças (activity log / histórico de domínio)

4.9 Relatórios, busca, duplicidade e LGPD

- PDF (DomPDF) e Excel/CSV (Maatwebsite) via jobs
- Busca global Postgres full-text (`tsvector` + GIN)
- Duplicidade sugerida: mesmo tipo + proximidade + janela temporal
- Retenção/anonimização, endpoint de esquecimento, consentimento e log de acesso

5. Fase 0 — Segurança e infraestrutura PO

Fundação sem a qual as demais fases não operam com segurança.

5.1 O que foi entregue

Item	Implementação
Posição do motorista unificada	<code>App\Services\DriverPositionService</code> consumido por controllers web e API
API de localização protegida	<code>auth:sanctum</code> + <code>throttle</code> ; sem aceitar <code>driver_id</code> arbitrário do payload
Canal privado	<code>routes/channels.php</code> : <code>occurrence.{id}</code> restrito por role ou motorista dono
Broadcasting real	<code>BROADCAST_DRIVER=pusher</code> (ou Socketi); evento <code>DriverPositionUpdated</code> chega ao front
Filas assíncronas	<code>QUEUE_CONNECTION=redis</code> + worker supervisionado
API autenticada	Endpoints sensíveis de ocorrências/tenants/tipos com <code>auth:sanctum</code>

5.2 Passo a passo de configuração

Passo 1 — Definir no `.env` : `BROADCAST_DRIVER` , `QUEUE_CONNECTION=redis` , `REDIS_*` , chaves Pusher/Soketi.

Passo 2 — Subir Redis e worker: `php artisan queue:work redis --tries=3`.

Passo 3 — Validar canais privados com dois usuários de roles diferentes.

Passo 4 — Confirmar `php artisan route:list` com middleware de auth nos endpoints corrigidos.

6. Fase 1 — Fundação de dados da ocorrência P1

6.1 Modelo de dados

Elemento	Descrição
<code>occurrences.protocol</code>	Único, gerado em <code>OccurrenceService::create</code>
<code>occurrences.priority_id</code>	FK para prioridades
<code>occurrences.due_at</code>	Calculado pelo SLA (tipo/prioridade)
Endereço estruturado	neighborhood, city, state, zipcode (+ address livre)
<code>priorities</code>	name, weight, color, default_sla_hours
<code>type_occurrences</code>	sla_hours, parent_id
<code>occurrence_status_history</code>	Histórico append-only de mudanças de status

6.2 Passo a passo funcional

Passo 1 — Atendente cria ocorrência: sistema gera protocolo e calcula `due_at`.

Passo 2 — Prioridade e tipo alimentam peso visual e prazo de SLA.

Passo 3 — Toda mudança de status grava linha em `occurrence_status_history`.

Passo 4 — Telas create/edit/show exibem protocolo (readonly), prioridade, endereço e timeline.

7. Fase 2 — RBAC expandido + Departamento/Equipe P1

7.1 Decisões implementadas

- Motorista = subtipo de Agente de Campo (reusa `drivers`, `ensure.driver` e GPS).
- Policies nativas Laravel sobre ACL caseiro (Role/Permission).
- Menus AdminLTE filtrados por permissão para Atendente/Supervisor.
- CRUD básico de departamentos e equipes.

7.2 Passo a passo de acesso

Passo 1 — Rodar seeders de roles/permissões.

Passo 2 — Vincular usuário a Role e, se agente, a registro em `drivers` (+ team opcional).

Passo 3 — Policies autorizam ações em Occurrence, Driver, Report etc.

Passo 4 — Administrador deixa de depender de e-mail hardcoded em `config/tenant.php`.

8. Fase 3 — Fluxo de status, auditoria, evidências e notificações P1 / P2

P2

8.1 Máquina de status

`OccurrenceService::updateStatus()` valida o mapa de-para. Transições inválidas são rejeitadas. Status terminais não permitem avanço.

8.2 Evidências antes/depois

- Campos em `occurrences_imagens`: `phase` (antes/depois), `lat/lng`, `captured_at`, `uploaded_by_user_id`
- Agente envia fotos no deslocamento/atendimento; admin visualiza na ficha

8.3 Notificações internas

- Canal database do Laravel
- Eventos: ocorrência atribuída, status alterado, SLA em risco
- Processadas pela fila Redis (Fase 0)

8.4 Passo a passo do atendimento

Passo 1 — Ocorrência entra como Recebida/Triagem.

Passo 2 — Supervisor atribui agente → status Aguardando aceitação + notificação.

Passo 3 — Agente aceita → Em deslocamento → Em atendimento, enviando evidências.

Passo 4 — Conclusão/Finalização registra timeline completa para auditoria e relatórios.

9. Fase 4 — Dashboard, mapa e busca global P1 / P2

9.1 Dashboard

- `DashboardService` agrega KPIs e séries para Chart.js
- Cards: abertas, em atendimento, finalizadas hoje, SLA em risco, SLA estourado

9.2 Mapa operacional

- Clustering com `@googlemaps/markerclusterer`
- Heatmap com `google.maps.visualization.HeatmapLayer`
- Filtros reutilizam padrão `/ {recurso} / search`
- Atualização por broadcasting; polling permanece como fallback

9.3 Busca global

Full-text search nativo do Postgres (`tsvector` + índice GIN) sobre protocolo, endereço, descrição e dados correlatos — sem Elasticsearch/Meilisearch.

10. Fase 5 — Despacho inteligente + GPS robusto P2

10.1 Ranking de agentes

```
-- Conceito da query (haversine em SQL Postgres)
SELECT drivers.*, (
  6371 * acos(
    cos(radians(:lat)) * cos(radians(lat)) *
    cos(radians(lng) - radians(:lng)) +
    sin(radians(:lat)) * sin(radians(lat))
  )
) AS distance_km
FROM drivers
WHERE status = 'disponivel'
  AND (team_id = :team_id OR :team_id IS NULL)
ORDER BY distance_km ASC
LIMIT 10;
```

10.2 Comportamento entregue

- Sistema sugere ranking; humano confirma a atribuição
- Status do Driver sincroniza com status da Occurrence no Service
- `driverRoute()` com limite de pontos
- Job de retenção remove posições antigas (> 48h)

10.3 Passo a passo do despacho

Passo 1 — Supervisor abre ocorrência e solicita sugestão de agentes.

Passo 2 — Sistema lista candidatos por distância/especialidade.

Passo 3 — Supervisor confirma o agente escolhido.

Passo 4 — GPS do agente alimenta o mapa admin em tempo real.

11. Fase 6 — Relatórios, duplicidade e LGPD P2 / P3

11.1 Exportações

Formato	Biblioteca	Uso
PDF	barryvdh/laravel-dompdf	Ficha da ocorrência e fechamento
Excel/CSV	maatwebsite/excel ^3	Listagens e tempo médio por etapa

Geração sempre via job/fila — a request apenas enfileira e notifica quando o arquivo está pronto.

11.2 Duplicidade

- Heurística: mesmo tipo + proximidade geográfica + janela de tempo
- Marca “possível duplicata” para revisão humana (sem merge automático)
- Opcional: similaridade textual com `pg_trgm`

11.3 LGPD

- Retenção/anonimização de `driver_positions` e dados pessoais antigos
- Endpoint de esquecimento
- Consentimento no formulário público
- Log de acesso a dados sensíveis

12. Passo a passo do ambiente de desenvolvimento

Passo 1 — Clonar o repositório e entrar em `flysop/`.

Passo 2 — Copiar `.env.docker.example` para `.env` (Postgres local: host `pgsql`, user/senha `laravel / secret`).

Passo 3 — No WSL: `docker compose -f docker-compose.dev.yml up -d --build`.

Passo 4 — Aguardar healthchecks de `pgsql/mysql` e boot do app (`artisan serve` na porta 8000).

Passo 5 — Acessar preferencialmente `http://127.0.0.1:8000` (evita problemas de IPv6/localhost no Windows+WSL).

Passo 6 — Rodar seeds: `docker compose -f docker-compose.dev.yml exec app php artisan db:seed`.

Passo 7 — Garantir Redis + worker para filas e broadcasting configurado no `.env`.

12.1 Serviços locais

Serviço	URL / Porta
Aplicação Laravel	http://127.0.0.1:8000
Nginx	http://127.0.0.1:80
PostgreSQL	localhost:5432
MySQL	localhost:3306
MinIO Console	http://127.0.0.1:8900

Estabilidade no Windows/WSL: use `vmIdleTimeout=-1` e, se necessário, `networkingMode=mirrored` em `%USERPROFILE%\wslconfig` para evitar queda do Docker e falha de `localhost` via IPv6.

13. Passo a passo operacional (uso do sistema)

13.1 Fluxo ponta a ponta

Passo 1 — Abertura: cidadão/atendente registra ocorrência (protocolo + prioridade + endereço).

Passo 2 — Triagem: supervisor classifica tipo/subtipo e avalia possíveis duplicatas.

Passo 3 — Despacho: sistema sugere agentes próximos; supervisor confirma.

Passo 4 — Aceite: agente aceita no painel motorista e inicia deslocamento (GPS ativo).

Passo 5 — Atendimento: evidências antes/depois, mudanças de status com timeline.

Passo 6 — Fechamento: status final + exportação PDF/Excel se necessário.

Passo 7 — Governança: dashboard de SLA, auditoria e rotinas LGPD.

13.2 Onde cada pessoa trabalha

Persona	Tela	Ações principais
Admin	/admin	Cadastros, prioridades, usuários, relatórios, LGPD
Supervisor	/admin/dashboard, ocorrências, mapa	Triagem, despacho, SLA
Atendente	/admin/occurrences	CRUD de ocorrências e anexos
Agente	/driver	Aceitar, GPS, evidências, status
Cidadão	site / API	Abrir e acompanhar protocolo

14. Checklist de verificação (aceite)

Fase	Como validar	Resultado esperado
0	<code>route:list</code> , teste de canal privado, job na fila	Auth ok; broadcast e queue funcionando
1–3	Criar ocorrência e mudar status	Protocolo, due_at, histórico e notificações
4	Abrir dashboard e mapa	KPIs, gráficos, cluster/heatmap e busca
5	Despachar com 2+ agentes	Ranking por distância e GPS ao vivo
6	Solicitar PDF/Excel e marcar duplicata	Arquivo via fila; sugestão humana de duplicidade

14.1 Bibliotecas adotadas

Necessidade	Escolha
Gráficos	Chart.js
PDF	barryvdh/laravel-dompdf
Excel	maatwebsite/excel ^3
Fila	Redis + queue:work
Broadcast	Pusher / Soketi
Cluster/Heatmap	MarkerClusterer + HeatmapLayer
Distância	Haversine SQL (upgrade opcional cube/earthdistance)
Busca	Postgres full-text
RBAC	Policies Laravel + ACL existente

Documento gerado a partir do plano **virtual-jingling-cookie.md** e da documentação do repositório (`docs/specs/*` , `ARQUITETURA_MOTORISTA.md` , `README.DOCKER.md`). Apresenta o produto na forma entregue das fases 0–6 da Central Inteligente de Ocorrências Públicas.